

Operator Overloading in C++

Get personalized course recommendations

Answer a few quick questions about your interests and skill level. We'll create a custom list of courses just for you.

[Take the quiz](#)

C++ Operator Overloading

C++ supports operator overloading, enabling you to redefine the way operators work with user-defined types. This allows operators like `+`, `-`, or `*` to be used with objects of classes you create, enhancing the versatility of your code.

C++ Operator Overloading Functions

Operator overloading in C++ happens through special member functions with specific names and return types. This technique allows you to define the behavior of operators for user-defined data types, thus extending the language's capabilities.

C++ `const` Keyword

In C++, when overloading operators, you can use the `const` keyword to ensure they work correctly with `const` objects. By marking a member function as `const`, you guarantee it doesn't modify any member variables. This is important for maintaining object immutability and integrity when dealing with read-only objects.

```
class Example {
public:
    // Overloaded operator with const
    keyword
    int operator[](size_t index) const {
        return array[index]; // No
modification permitted
    }

private:
    int array[5] {1, 2, 3, 4, 5};
};

int main() {
    const Example example;
    int value = example[2]; // Access
through const operator
    return 0;
}
```

Overloading Arithmetic Operators

In C++, arithmetic operators like `+`, `-`, `*`, `/`, and `%` can be overloaded to perform computations on user-defined types, such as classes. This allows you to define how objects of these types interact mathematically, enhancing code flexibility and readability.

```
class Complex {
public:
    double real, imag;
    Complex(double r, double i) : real(r),
    imag(i) {}

    // Overloading the + operator
    Complex operator + (const Complex&
other) {
        return Complex(real + other.real,
imag + other.imag);
    }
};

int main() {
    Complex a(1.0, 2.0);
    Complex b(3.0, 4.0);
    Complex c = a + b; // Uses the
overloaded + operator
    return 0;
}
```

C++ Custom Comparisons

Leverage operator overloading in C++ to customize how objects are compared using operators like `==`, `!=`, `<`, `>`, `<=`, and `>=`. This feature enables developers to implement object-specific logic for comparison, enhancing flexibility in storing and manipulating complex data types.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;
    int grade;

    Student(string n, int g) : name(n),
grade(g) {}

    // Overloading the == operator
    bool operator== (const Student& rhs)
const {
        return this->grade == rhs.grade;
    }

    // Overloading the != operator
    bool operator!= (const Student& rhs)
const {
        return !(*this == rhs);
    }
};

int main() {
    Student student1("Alice", 85);
    Student student2("Bob", 85);

    if (student1 == student2) {
        cout << student1.name << " and " <<
student2.name << " have the same grade."
<< endl;
    } else {
        cout << student1.name << " and " <<
student2.name << " have different
grades." << endl;
    }

    return 0;
}
```

C++ Overloading Assignment Operator

In C++, assignment operators such as `=`, `+=`, and `-=` can be overloaded to define custom behavior. This allows for more intuitive and streamlined code when dealing with complex objects.

```
class Box {  
public:  
    int length;  
  
    Box(int len) : length(len) {}  
  
    Box& operator=(const Box &b) {  
        length = b.length;  
        return *this;  
    }  
};  
  
int main() {  
    Box box1(10);  
    Box box2(20);  
    box2 = box1; // Uses the overloaded  
assignment operator  
    return 0;  
}
```

C++ Overloading Prefix and Postfix Operators

In C++, you can overload the increment `++` and decrement `--` operators in both prefix and postfix forms. Overloading allows you to define custom behavior for these operators in your custom classes, enhancing flexibility and control over objects.

```
#include <iostream>

class Counter {
public:
    Counter() : value(0) {}

    // Prefix increment
    Counter& operator++() {
        ++value;
        return *this;
    }

    // Postfix increment
    Counter operator++(int) {
        Counter temp = *this;
        value++;
        return temp;
    }

    void display() const {
        std::cout << "Value: " << value <<
std::endl;
    }

private:
    int value;
};

int main() {
    Counter c;
    ++c;
    c.display(); // Displays: Value: 1

    c++;
    c.display(); // Displays: Value: 2

    return 0;
}
```

C++ Subscript Operator

Learn how to overload the subscript operator (`[]`) in C++ to grant custom container types array-like access. This is a powerful technique that allows you to provide intuitive access to elements in your data structures, mimicking native arrays or lists.

```
#include <iostream>

class CustomContainer {
    int arr[10];
public:
    CustomContainer() {
        for(int i = 0; i < 10; ++i) arr[i] = i;
    }
    // Overloading subscript operator
    int& operator[](int index) {
        if (index < 0 || index >= 10) {
            std::cout << "Index out of
bounds!\n";
            exit(EXIT_FAILURE);
        }
        return arr[index];
    }
};

int main() {
    CustomContainer container;

    // Modifies the element at index 2
    container[2] = 42;

    // Outputs: 42
    std::cout << container[2] << std::endl;
    return 0;
}
```

C++ Functors

A **functor** in C++ is an object acting like a function. By overloading the `operator()` within a class, an instance of this class behaves like a function call. This feature enables passing operations as objects, providing flexibility for callbacks or templated functions.

```
#include <iostream>

class Adder {
public:
    Adder(int n) : num(n) {}
    int operator()(int x) const {
        return num + x;
    }
private:
    int num;
};

int main() {
    Adder addFive(5);
    // Output 15
    std::cout << "5 + 10 = " << addFive(10)
<< std::endl;
    return 0;
}
```


C++ Conversion Operators

In C++, you can define conversion operators to change one type into another. Conversion can be **implicit** or **explicit**. Implicit conversion is automatic, while explicit conversion is intentionally invoked using `static_cast` or similar methods. **Conversion operators** enhance flexibility, creating seamless interactions between different types.

```
class Distance {
public:
    Distance(double km) : kilometers(km) {}

    // Conversion operator to convert
    // Distance to double implicitly
    operator double() const {
        return kilometers;
    }

private:
    double kilometers;
};

int main() {
    Distance dist(100.0);
    double kmDistance = dist; // Implicit
    // conversion using operator
    std::cout << "Distance: " << kmDistance
    << " km" << std::endl;
    return 0;
}
```

C++ Operator Overloading

Operator overloading in C++ allows customizing the way operators work with user-defined types. To maintain clear and understandable code, ensure that overloaded operators behave in a way that aligns with their traditional meanings. This helps avoid confusion for those reading or maintaining the code.

